5. Design Search Autocomplete/Typeahead

0) Requirements & Sizing

Functional

- Return **top 5-10 suggestions** as users type prefix queries
- Rank by popularity, recency, and optionally personalization
- Support **multi-language** queries (Unicode-aware)
- **Filter** offensive/inappropriate suggestions
- Handle **trending** queries (boost recent spikes)
- Optional: personalized suggestions based on user history

Non-functional

Metric	Target
Scale	~100K QPS (peak) for suggestions
p99 latency	< **50-100ms** (critical for UX)
Trie size	~~10M unique queries (~~1-2GB in memory)
Update frequency	Hourly batch + real-time trending
Availability	99.9% (suggestions are non-critical graceful degradation)
Global	Multi-region for low latency

Capacity

```
Assumptions:
- 10M unique searchable phrases
- Average phrase length: 20 characters
- Top-10 suggestions per node: 10 × 8 bytes = 80 bytes
- Trie overhead: ~50 bytes per node

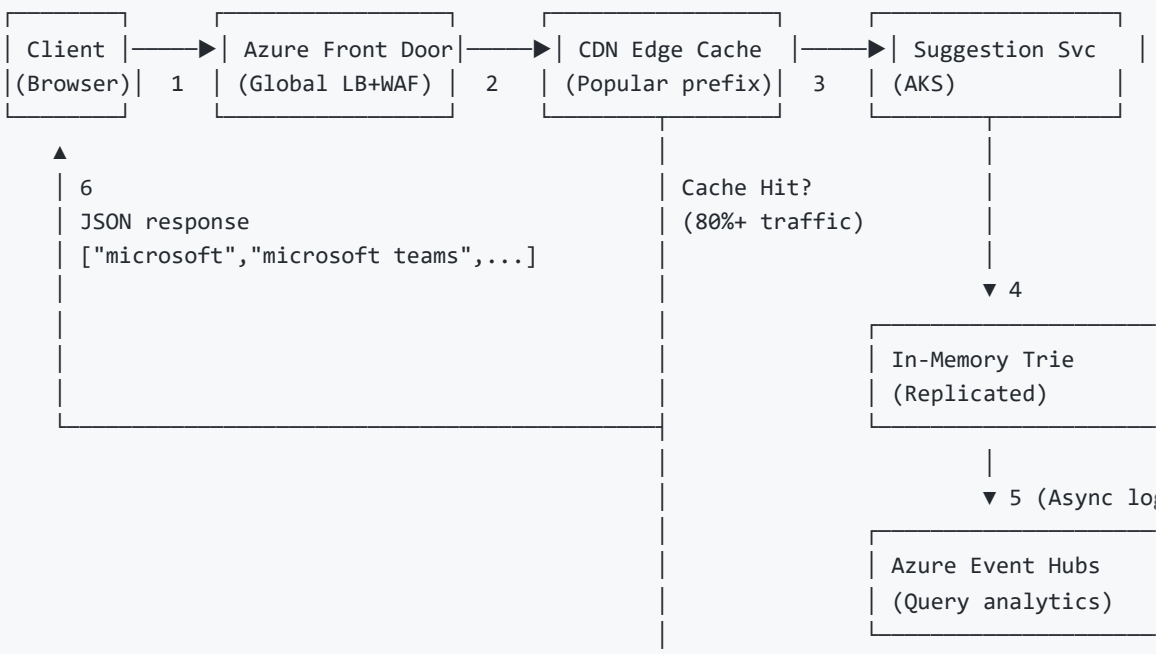
Memory estimate:
- 10M phrases × 20 chars = 200M nodes (worst case)
- Optimized trie with sharing: ~50M nodes
- 50M × 50 bytes = ~2.5GB per replica
```

```
With compression + only storing top suggestions:
- Practical size: **1-2GB per server**
```

---

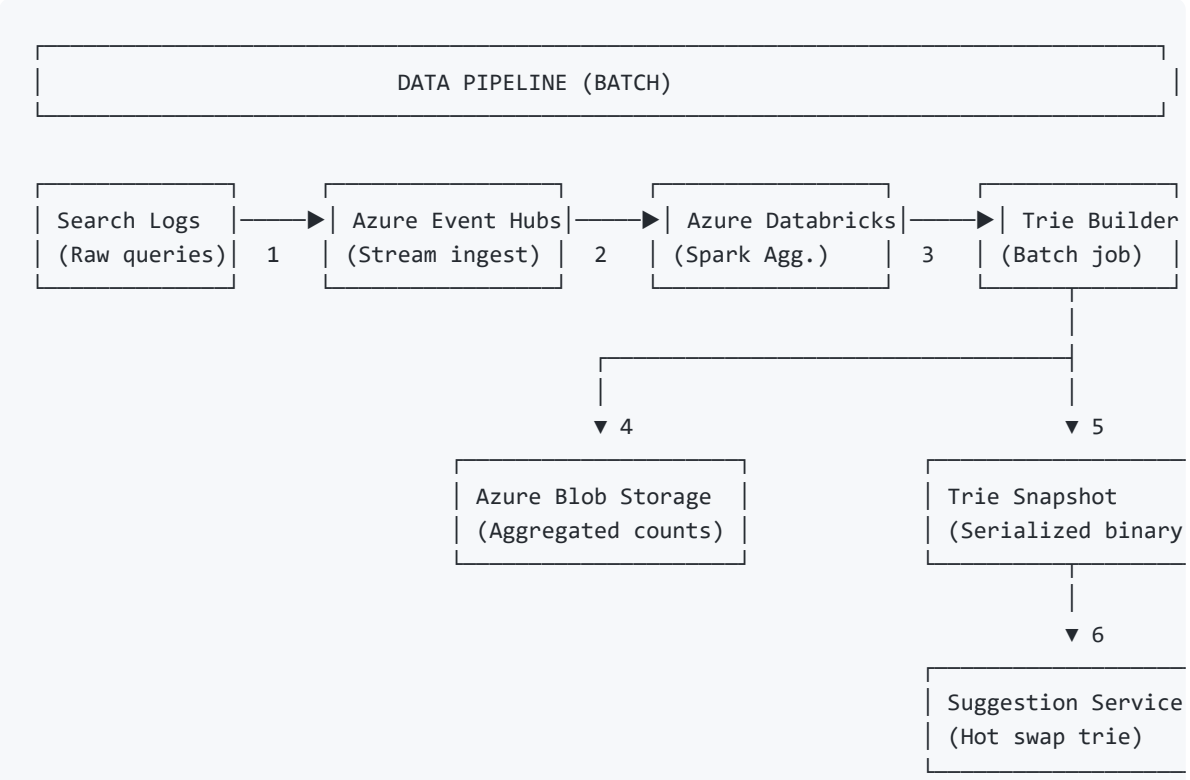
1) High-Level Architecture

Read Path (Get Suggestions)



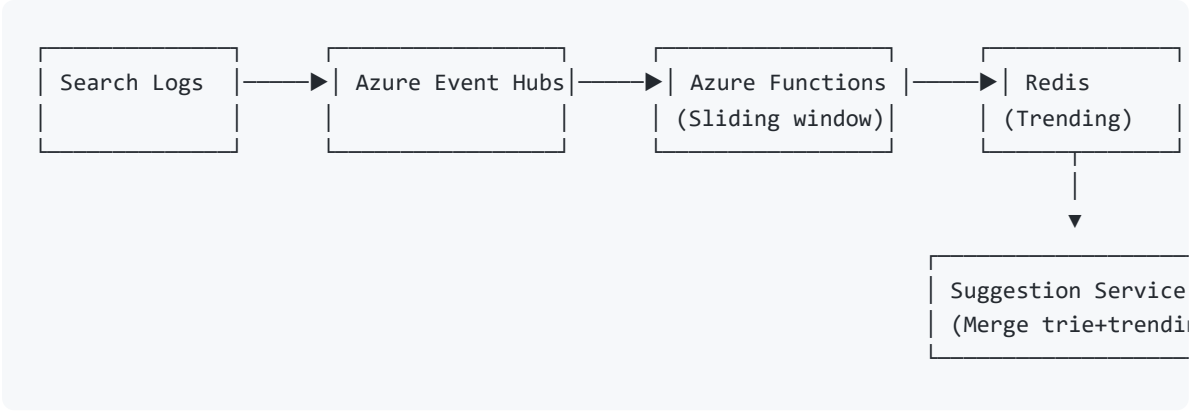
Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	GET `/suggest?q=micros` (debounced 100-200ms)	SYNC
2	Front Door	CDN Edge	Check edge cache for popular prefix	SYNC
3	CDN (miss)	Suggestion Service	Route to nearest AKS cluster	SYNC
4	Suggestion Service	In-Memory Trie	Lookup prefix → return pre-computed top-K	SYNC
5	Suggestion Service	Event Hubs	Log query for analytics (fire & forget)	**ASYNC**
6	Suggestion Service	Client	Return JSON array of suggestions	SYNC

Write Path (Update Trie)



Step	From	To	Action	Frequency
1	Search Service	Event Hubs	Stream completed search queries	Real-time
2	Event Hubs	Databricks	Consume and aggregate by time window	Continuous
3	Databricks	Trie Builder	Compute frequency scores, filter, rank	Hourly
4	Trie Builder	Blob Storage	Store aggregated phrase counts	Hourly
5	Trie Builder	Blob Storage	Serialize trie to binary snapshot	Hourly
6	Suggestion Service	Memory	Load new trie snapshot (hot swap)	Hourly

Real-Time Trending Path



> **Key insight**: Trending queries (last 1-4 hours) are computed in near real-time and merged with the static trie at query time.

Components Summary

Component	Azure Service	Purpose
Global Entry	**Azure Front Door**	Anycast DNS, WAF, route to nearest region
Edge Cache	**Azure CDN**	Cache popular prefixes (80%+ hit rate)
Compute	**Azure Kubernetes Service (AKS)**	Run suggestion service pods
In-Memory Store	**Application Memory**	Hold trie (~1-2GB per pod)
Trending Cache	**Azure Cache for Redis**	Real-time trending queries
Event Stream	**Azure Event Hubs**	Ingest search query logs
Batch Processing	**Azure Databricks**	Aggregate query frequencies
Storage	**Azure Blob Storage**	Trie snapshots, aggregated data
Filter Service	**Azure ML**	Offensive content detection

API Design

Core APIs

Method	Endpoint	Description
`GET`	`/api/v1/suggestions?q={prefix}&limit=10`	Get autocomplete suggestions
`POST`	`/api/v1/searches`	Log a completed search (for ranking updates)

Request/Response Examples

Get Suggestions:

```
GET /api/v1/suggestions?q=micro&limit=5

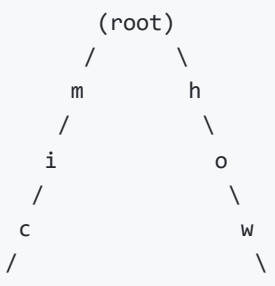
Response: 200 OK
{
  "suggestions": [
    { "text": "microsoft teams", "score": 9500 },
    { "text": "microservices", "score": 8200 },
    { "text": "microsoft azure", "score": 7800 },
    { "text": "microcontroller", "score": 3100 },
    { "text": "micro frontend", "score": 2900 }
  ],
  "query_time_ms": 12
}
```

Design Considerations

- Response time must be < **100ms** for good UX
- Use **HTTP GET** (cacheable) rather than POST
- Support **debouncing** on client side (200-300ms delay)
- Return results sorted by **popularity score**

2) Data Structure Options

✅ Recommended: Trie with Pre-computed Top-K



```
           r                    ├─ top-K: ["how to", "how are you", ...]
          /
         o
        / \
       s   n
      /
     o ─── top-K: ["microsoft", "microsoft teams", "microsoft office", ...]
    /
   f
  /
t ─── top-K: ["microsoft", "microsoft teams", ...]
```

```
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    List<Suggestion> topK = new ArrayList<>(10);  // Pre-computed!
    boolean isEndOfWord;
}

class Suggestion {
    String phrase;
    long frequency;
    double score;       // α*frequency + β*recency + γ*personalization
    long lastUpdated;
}
```

Why pre-compute top-K at each node?

- Without: O(prefix) + O(traverse entire subtree) = slow
- With: O(prefix) + O(1) = **fast** (<1ms lookup)

Alternative: Prefix Hash Map

```
Map<String, List<Suggestion>> prefixToSuggestions;
// "mic" → ["microsoft", "microwave", ...]
// "micr" → ["microsoft", "microwave", ...]
// "micro" → ["microsoft", "microwave", ...]
```

Aspect	Trie	Prefix Hash Map
Memory	More efficient (shared prefixes)	Redundant storage
Lookup	O(prefix length)	O(1) hash lookup
Updates	Complex (bubble up changes)	Simple (regenerate)
Memory for 10M phrases	~1-2GB	~5-10GB

Recommendation: **Trie** for memory efficiency, **Hash Map** if simplicity > memory.

❌ Rejected: Database Lookup

- Too slow for <50ms p99
- Even with Redis, serialization overhead adds latency

---

3) Data Model

Query Aggregation (Blob Storage / Data Lake)

```
{
  "phrase": "microsoft teams",
  "frequency_1h": 15420,
  "frequency_24h": 245000,
  "frequency_7d": 1850000,
  "last_searched": "2026-01-27T08:00:00Z",
  "language": "en",
  "category": "technology",
  "is_safe": true
}
```

Trie Snapshot Format (Binary)

```
Header:
  - Version: uint32
  - Node count: uint32
  - Phrase count: uint32

Nodes (BFS order):
  - Character: uint16 (Unicode)
  - Children offset: uint32
  - Top-K offset: uint32
  - IsEndOfWord: bool

Suggestions:
  - Phrase offset: uint32
  - Score: float32

Strings (compressed):
  - Null-terminated UTF-8 strings
```

Redis Trending Cache

```
Key: trending:{region}:{hour}
Type: Sorted Set
Score: frequency in last hour
Value: query phrase

ZADD trending:westus:2026012708 15420 "microsoft teams"
ZREVRANGE trending:westus:2026012708 0 99  // Get top 100
```

4) Read Path Detail





5) Ranking Algorithm

Score Calculation

```
Score = α × frequency + β × recency + γ × personalization + δ × trending

Where:
  α = 0.5  (base popularity weight)
```

```
β = 0.2  (recency decay factor)
γ = 0.2  (user personalization, if logged in)
δ = 0.1  (trending boost)
```

Frequency Weighting

```
double frequencyScore(long freq7d) {
    // Log scale to prevent popular queries from dominating
    return Math.log10(freq7d + 1);
}
```

Recency Decay

```
double recencyScore(long lastSearchedTs, long nowTs) {
    long hoursAgo = (nowTs - lastSearchedTs) / 3600000;
    // Exponential decay: half-life of 24 hours
    return Math.exp(-0.029 * hoursAgo);
}
```

Trending Boost

```
double trendingScore(long freqLastHour, long avgHourlyFreq) {
    if (avgHourlyFreq == 0) return 0;
    double ratio = (double) freqLastHour / avgHourlyFreq;
    // Boost if 2x+ normal traffic
    return ratio > 2.0 ? Math.log(ratio) : 0;
}
```

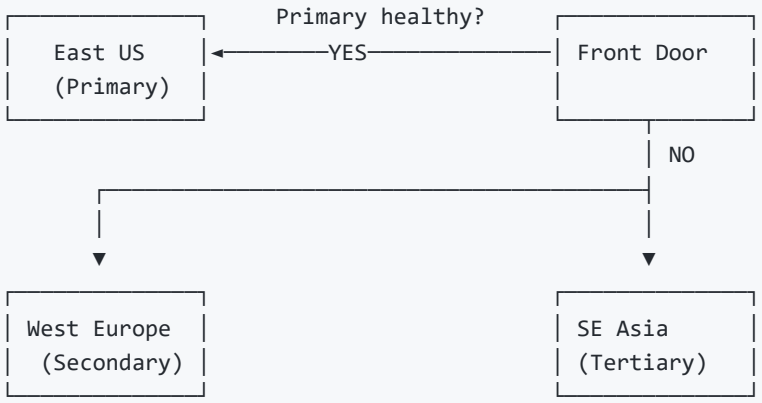
---

6) Multi-Region & DR

```
Active-Active: East US, West Europe, Southeast Asia
  │
  ├── Front Door anycast → nearest healthy region
  ├── Each region has:
  │     ├── Full trie replica (loaded from same Blob snapshot)
  │     ├── Regional Redis (trending cache)
  │     └── CDN edge nodes
  │
```

```
   ├── Trie updates: Push snapshot to all regions simultaneously
   └── Trending: Regional (no cross-region sync needed)
```

Failover Strategy



---

7) Security & Governance

Concern	Solution
Offensive content	ML-based filter + blocklist; review flagged suggestions
Query injection	Sanitize input, limit length (100 chars max)
PII in suggestions	Never include email, phone, SSN patterns
Abuse (scraping)	Rate limit by IP (100 req/min), CAPTCHA for suspicious
Logging PII	Hash/truncate user IDs, don't log full queries with PII
GDPR	Allow users to opt-out of personalization

Input Validation

```
String sanitizePrefix(String input) {
    if (input == null || input.length() > 100) {
        throw new BadRequestException("Invalid prefix");
    }
    // Remove control characters, normalize Unicode
    return Normalizer.normalize(input, Normalizer.Form.NFC)
                     .replaceAll("[\\p{Cntrl}]", "")
                     .toLowerCase()
```

```
                    .trim();
}
```

---

8) Failure Modes

Failure	Impact	Mitigation
Trie service down	No suggestions	CDN serves stale; client shows "no suggestions" gracefully
Redis (trending) down	No trending boost	Fall back to static trie only (still functional)
Trie build fails	Stale suggestions	Keep serving previous snapshot; alert on-call
CDN outage	Higher origin load	Front Door routes directly; auto-scale AKS
Event Hubs down	Missing analytics	Suggestions still work; backfill from logs

Graceful Degradation

```
List<Suggestion> getSuggestions(String prefix) {
    try {
        List<Suggestion> results = trie.lookup(prefix);
        try {
            results = mergeWithTrending(results, redis.getTrending(prefix));
        } catch (RedisException e) {
            log.warn("Trending unavailable, using static trie");
        }
        return results;
    } catch (Exception e) {
        log.error("Trie lookup failed", e);
        return Collections.emptyList();  // Graceful degradation
    }
}
```

---

9) Cost Estimates

Component	Sizing	Monthly Cost (Est.)

AKS	6 nodes × Standard_D4s_v3 (3 regions × 2 for HA)	~$1,500
Azure CDN	100TB egress, 1B requests	~$3,000
Redis	P1 (6GB) × 3 regions	~$750
Event Hubs	10 TUs, 7-day retention	~$700
Databricks	4 DBU-hours/day for batch	~$400
Blob Storage	100GB snapshots + logs	~$50
Front Door	1B requests	~$400
Total		**~$6,800/month**

---

10) Trade-offs Discussion

Decision	Option A	Option B	Choice	Reasoning
Data structure	Trie	Hash Map	**Trie**	5x memory savings for large vocabularies
Top-K storage	Pre-computed	On-demand	**Pre-computed**	O(1) vs O(subtree traversal) at query time
Update model	Real-time	Batch	**Batch + trending**	Simpler, lower cost; trending handles spikes
Personalization	Built-in	Separate	**Separate service**	Keeps suggestion service stateless
CDN caching	Aggressive	Conservative	**Aggressive (5-15min)**	80%+ hit rate worth slight staleness

---

11) 60-Second Explanation

> "Typeahead is a **read-heavy, latency-critical** system. I use an **in-memory Trie** with **pre-computed top-K suggestions at each node** for O(prefix-length) + O(1) lookups.

> The trie is built **hourly** from search logs aggregated via **Event Hubs → Databricks → Blob Storage**. Each suggestion service pod loads the same snapshot (~1-2GB).
>
> For **real-time trending**, I use a **Redis sorted set** with sliding window counts, merged at query time.
>
> **CDN caches 80%+ of requests** (popular prefixes like 'how to'). Client-side **debouncing** (100-200ms) reduces load by 5-10x.
>
> The system is **stateless** and **horizontally scalable**. With multi-region deployment behind **Front Door**, we achieve **<50ms p99** and handle **100K+ QPS** at peak."

Technologies Deep Dive

Technology	Role	Reference
Trie	Prefix lookup data structure	Tech Guide
Redis	Trending cache, sorted sets	Tech Guide
Kafka/Event Hubs	Query log streaming	Tech Guide
CDN	Edge caching for popular prefixes	Tech Guide
Bloom Filter	Quick prefix existence check	Tech Guide

Appendix: Trie Implementation

```
public class TypeaheadTrie {
    private final TrieNode root = new TrieNode();
    private static final int TOP_K = 10;

    static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        List<Suggestion> topSuggestions = new ArrayList<>();
        boolean isEndOfWord;
    }

    static class Suggestion implements Comparable<Suggestion> {
        String phrase;
        double score;

        @Override
        public int compareTo(Suggestion o) {
            return Double.compare(o.score, this.score); // Descending
```

```
        }
    }

    // Build trie with pre-computed top-K at each node
    public void build(List<Suggestion> allSuggestions) {
        // Sort all suggestions by score descending
        allSuggestions.sort(Comparator.comparingDouble(s -> -s.score));

        for (Suggestion suggestion : allSuggestions) {
            TrieNode node = root;
            for (char c : suggestion.phrase.toCharArray()) {
                node = node.children.computeIfAbsent(c, k -> new TrieNode());
                // Add to top-K if not full or better than worst
                if (node.topSuggestions.size() < TOP_K) {
                    node.topSuggestions.add(suggestion);
                }
            }
            node.isEndOfWord = true;
        }
    }

    // O(prefix length) lookup
    public List<Suggestion> getSuggestions(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            node = node.children.get(c);
            if (node == null) {
                return Collections.emptyList();
            }
        }
        return node.topSuggestions;
    }
}
```

Appendix: Client-Side Debouncing

```
// Debounce to avoid flooding server on every keystroke
function debounce(func, wait) {
    let timeout;
    return function executedFunction(...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() => func.apply(this, args), wait);
    };
}

const fetchSuggestions = debounce(async (prefix) => {
    if (prefix.length < 2) return; // Min 2 chars

    // Check local cache first
    const cached = localStorage.getItem(`suggest:${prefix}`);
```

```
    if (cached) {
        const { suggestions, ts } = JSON.parse(cached);
        if (Date.now() - ts < 300000) { // 5 min cache
            renderSuggestions(suggestions);
            return;
        }
    }

    const response = await fetch(`/suggest?q=${encodeURIComponent(prefix)}`);
    const suggestions = await response.json();

    // Cache locally
    localStorage.setItem(`suggest:${prefix}`,
        JSON.stringify({ suggestions, ts: Date.now() }));

    renderSuggestions(suggestions);
}, 150); // 150ms debounce

searchInput.addEventListener('input', (e) => fetchSuggestions(e.target.value));
```